

CodeSys Modbus Serial Communication with Watlow F4S Temperature Controller Investigation Log

Summary: -

Following the successful linux modbus proof of concept moving on to

the investigation which is the end-to-end log of communication establishment between CODESYS and Modbus COM Port of Watlow F4S Temperature Controller

Target Hardware: Watlow F4S (SN 038983), Modbus RTU / RS-232, /dev/ttyWatlowF4S → COM2

CODESYS Runtime: CODESYS Control for Linux ARM64 SL on Raspberry Pi (IP 10.1.6.17)

Status: Live troubleshooting session with all terminal commands and outputs recorded below

Phase 0: Network Stabilization (Mandatory Prerequisite)

Problem discovered: Raspberry Pi's kernel packet-batching optimizations (TSO/GSO/GRO) cause TX stalls, interfering with both EtherCAT and Modbus latency.

[115441.794278] macb 1f00100000.ethernet internet_port: TX stall detected on queue 0

(tail=1437164 head=1437166); re-kicking TSTART

0.1 — Identify the ethernet port

bash

ip a

Actual output from the session: -

```
5: internet.6@internet_port: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 2c:cf:67:b9:b7:95 brd ff:ff:ff:ff:ff:ff
    inet 10.1.6.17/24 brd 10.1.6.255 scope global noprefixroute internet.6
        valid_lft forever preferred_lft forever
    inet6 fe80::3f93:72a3:b16e:bb7c/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Key observation: Port name is **internet_port**, NOT eth0. This was the Beckhoff DLS008 EtherCAT controller variant.

0.2 — Check current offloading status

bash

```
ethtool -k internet_port | grep -E "tcp-segmentation|generic-segmentation|generic-receive"
```

Before fix:

tcp-segmentation-offload: off [requested on]

generic-segmentation-offload: off [requested on]

generic-receive-offload: on

Problem: Two offloads show [requested on] as the kernel keeps trying to re-enable them.

0.3 — Disable offloading (applied INDIVIDUALLY, not combined)

```
mechatronics@LeftHandSmallTempCab:~/ssh/Temperature-Cabinet-Setpoint-Control-from-CODESYS-HMI $ bashsudo ethtool -K internet_port tso off
sudo ethtool -K internet_port gso off
sudo ethtool -K internet_port gro off
```

Why individually? Applying multiple flags in one command can cause the kernel to ignore the entire call if one flag is invalid. Single commands guarantee each one processes.

0.4 — Verify the change

bash

```
ethtool -k internet_port | grep -E "tcp-segmentation|generic-segmentation|generic-receive"
```

After fix:

```
mechatronics@leftHandSmallTempCab:~/ssh/Temperature-Cabinet-Setpoint-Control-from-CODESYS-HMI $ ethtool -k internet_port | grep -E "tcp-segmentation|generic-segmentation|generic-receive"
tcp-segmentation-offload: off
    tx-tcp-segmentation: off
generic-segmentation-offload: off
generic-receive-offload: off
```

All three show off (no [requested on] anymore).

0.5 — Make it permanent (survives reboot)

bash

```
sudo nano /etc/network/if-up.d/disable-offloading
```

Paste:

```
GNU nano 8.4 /etc/network/if-up.d/disable-offloading
#!/bin/bash
ethtool -K internet_port tso off
ethtool -K internet_port gso off
ethtool -K internet_port gro off
```

Save (Ctrl+O, Enter, Ctrl+X)

bash

```
sudo chmod +x /etc/network/if-up.d/disable-offloading
```

This script runs automatically on every network interface up event (including boot).

Phase 1: Linux Layer — udev Symlink + SysCom Config

Problem: /dev/ttyUSB0 is not stable. Replugging the adapter or rebooting can reassign it to /dev/ttyUSB1, silently breaking CODESYS config with no error message.

Solution: Create a permanent udev symlink tied to the adapter's hardware ID (vendor/product), which never changes.

1.1 — Identify the adapter's hardware ID

Bash `udevadm info -a -n /dev/ttyUSB0 | grep -E "idVendor|idProduct" | head -2`

Actual output:

```
mechatronics@LeftHandSmallTempCab:~/ssh/Temperature-Cabinet-Setpoint-Control-from-CODESYS-HMI $ udevadm info -a -n /dev/ttyUSB0 | grep -E "idVendor|idProduct" | head -2
ATTRS{idProduct}=="2303"
ATTRS{idVendor}=="067b"
```

These are the Prolific PL2303 hardware IDs which **fixed to the physical adapter**.

1.2 — Create the udev rule

bash

echo `'SUBSYSTEM=="tty", ATTRS{idVendor}=="067b", ATTRS{idProduct}=="2303", SY`

Critical detail: `SUBSYSTEM=="tty"` must come first as this tells udev "only process entries in the /dev/tty* tree, not the whole USB bus." Without it, the symlink points to a USB bus path (wrong) instead of ttyUSB0 (correct).

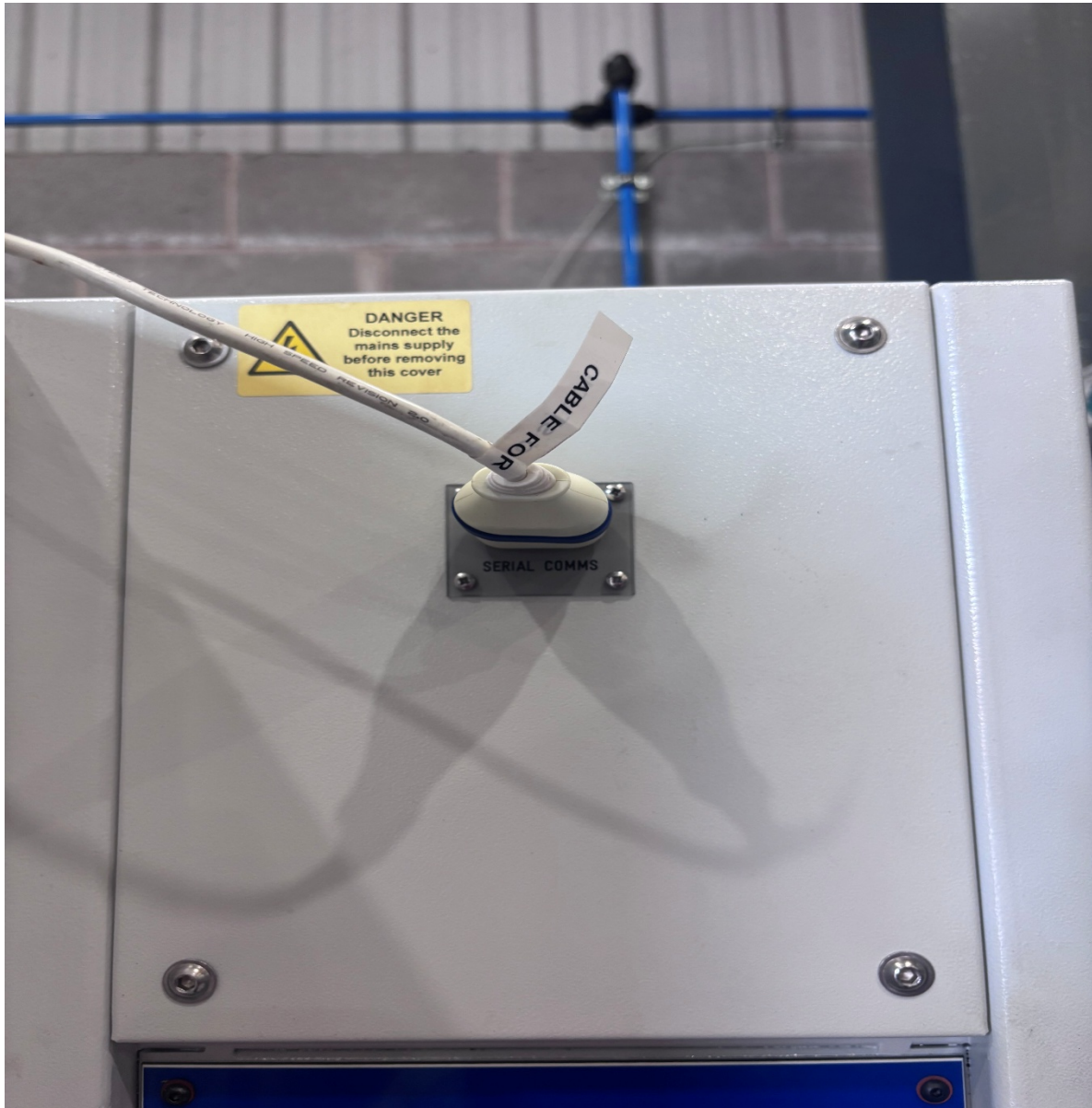
1.3 — Reload udev and trigger

```
mechatronics@LeftHandSmallTempCab:~/ssh/Temperature-Cabinet-Setpoint-Control-from-CODESYS-HMI $ sudo udevadm control --reload-rules
sudo udevadm trigger
```


1.4 — Unplug and replug the adapter

Physical action: disconnect USB adapter from Raspberry Pi, wait 2 seconds, reconnect.





1.5 — Verify the symlink

bash

ls -la /dev/ttyWatlowF4S

Expected: -

```
mechatronics@LeftHandSmallTempCab:~/ssh/Temperature-Cabinet-Setpoint-Control-from-CODESYS-HMI $ ls -la /dev/ttyWatlowF4S
lrwxrwxrwx 1 root root 7 Jul 15 14:14 /dev/ttyWatlowF4S -> ttyUSB0
```

1.6 — Update SysCom config to use stable name for CODESYS


```
cat /etc/CODESYSControl_User.cfg | grep SysCom -A 4
```

Look at which COM Port is CODESYS pointing towards :-

Expected Output :-

```
mechatronics@LeftHandSmallTempCab:~/ssh/Temperature-Cabinet-Setpoint-Control-from-CODESYS-HMI $ cat /etc/CODESYSControl_User.cfg | grep SysCom -A 4
[SysCom]
Linux.Devicefile.2=/dev/ttyWatlowF4S
portnum.2=2
baudrate.2=19200
```

Imp – CODESYS is pointing towards USB COM Port 2 as portnum 2=2 and baudrate 2 = 19200 same as on the temperature cabinet which is already observed in remote setpoint investigation log for temperature cabinet



Key change: Point to /dev/ttyWatlowF4S (stable symlink) instead of /dev/ttyUSB0 (unreliable).

Save (Ctrl+O, Enter, Ctrl+X).

1.8 — Restart CODESYS runtime

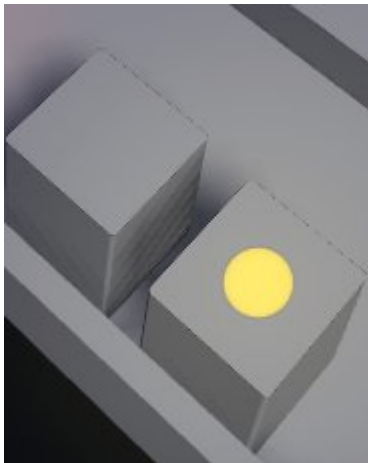
bash

sudo systemctl restart codesyscontrol

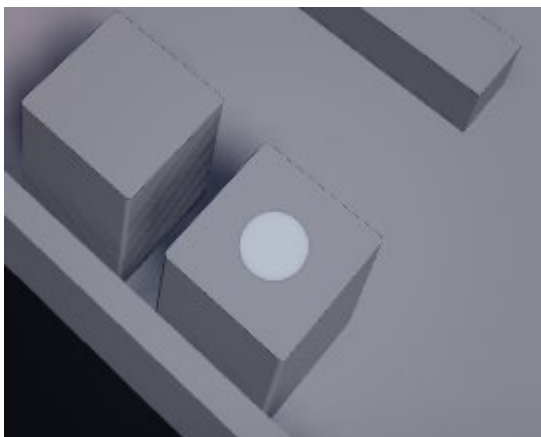
sudo systemctl status codesyscontrol

Expected Output: -

When it restarts on the 3D Rig Overview



When attempting to restart on 3D Rig Overview: -



CODESYS Status Expected:-

```
mechatronics@LeftHandSmallTempCab:~/ssh/Temperature-Cabinet-Setpoint-Control-from-CODESYS-HMI $ sudo systemctl start codesyscontrol
sudo systemctl status codesyscontrol
lines 1-14
● codesyscontrol.service - LSB: Prepares and starts codesyscontrol
   Loaded: loaded (/etc/init.d/codesyscontrol; generated)
   Active: active (running) since Wed 2026-07-15 08:36:24 BST; 25ms ago
 Invocation: be4afe9be54447e8b15cc2546919efe4
    Docs: man:systemd-sysv-generator(8)
  Process: 179889 ExecStart=/etc/init.d/codesyscontrol start (code=exited, status=0/SUCCESS)
    Tasks: 2 (limit: 9621)
      CPU: 197ms
   CGroup: /system.slice/codesyscontrol.service
           └─179893 /opt/codesys/bin/codesyscontrol.bin /etc/codesyscontrol/CODESYSControl.cfg

Jul 15 08:36:23 LeftHandSmallTempCab systemd[1]: Starting codesyscontrol.service - LSB: Prepares and starts codesyscontrol...
```

Runtime is running cleanly, no errors in journalctl.

1.9 — Baseline check (Linux layer only)

Before touching CODESYS, verify mbpoll still works on the stable symlink:

bash

mbpoll -m rtu -a 1 -b 19200 -P none -t 4 -r 100 -c 1 -1 -0 /dev/ttyWatlowF4S

Expected Output: -

1.For actual cabinet temperature: -

```
mechatronics@LeftHandSmallTempCab:~/ssh/Temperature-Cabinet-Setpoint-Control-from-CODESYS-HMI $ mbpoll -m rtu -a 1 -b 19200 -P none -t 4 -r 100 -c 1 -1 -0 /dev/ttyMatlowF45
This program comes with ABSOLUTELY NO WARRANTY.
This is free software, and you are welcome to redistribute it
under certain conditions; type 'mbpoll -w' for details.

Protocol configuration: ModBus RTU
Slave configuration...: address = [1]
                        start reference = 100, count = 1
Communication.....: /dev/ttyMatlowF45, 19200-8N1
                        t/o 1.00 s, poll rate 1000 ms
Data type.....: 16-bit register, output (holding) register table

-- Polling slave 1...
[100]: 580
```

Register 100 (temperature) returns 580 (58.0°C).

2.For current setpoint: -

```
mechatronics@LeftHandSmallTempCab:~/ssh/Temperature-Cabinet-Setpoint-Control-from-CODESYS-HMI $ mbpoll -m rtu -a 1 -b 19200 -P none -t 4 -r 300 -c 1 -1 -0 /dev/ttyMatlowF45
Copyright (c) 2015-2023 Pascal JEAN, https://github.com/epsilon-rt/mbpoll
This program comes with ABSOLUTELY NO WARRANTY.
This is free software, and you are welcome to redistribute it
under certain conditions; type 'mbpoll -w' for details.

Protocol configuration: ModBus RTU
Slave configuration...: address = [1]
                        start reference = 300, count = 1
Communication.....: /dev/ttyMatlowF45, 19200-8N1
                        t/o 1.00 s, poll rate 1000 ms
Data type.....: 16-bit register, output (holding) register table

-- Polling slave 1...
[300]: 250
```

Register 300 (setpoint) returns 250 (25.0°C).

3.Observed on F4S Front Panel: -



Phase 2: CODESYS Device Tree (On IDE)

2.1 — Create data types

In CODESYS IDE, project tree → **DataTypes** folder (right-click → **Add Data Type**).

E_SetpointState:

```
1      {attribute 'qualified_only'}
2      {attribute 'strict'}
3
4      TYPE E_SetpointState :
5      (
6          IDLE           := 0,
7          READY          := 10,
8          WRITING        := 20,
9          CONFIRM        := 30,
10         FAULTED        := 99
11     ) INT;
12     END_TYPE
13
```

E_FaultCode:


```

1      {attribute 'qualified_only'}
2      {attribute 'strict'}
3
4      TYPE E_FaultCode :
5      (
6          NO_FAULT           := 0,
7          COMMS_TIMEOUT      := 1,
8          WRITE_FAILED       := 2,
9          NOT_ACCEPTED       := 3,
10         RANGE_LOW          := 4,
11         RANGE_HIGH         := 5,
12         OVER_TEMP          := 6
13     ) INT;
14     END_TYPE
15

```

2.2 — Create GVL_Modbus

Project tree → GlobalVariableLists (right-click → Add Global Variable List):

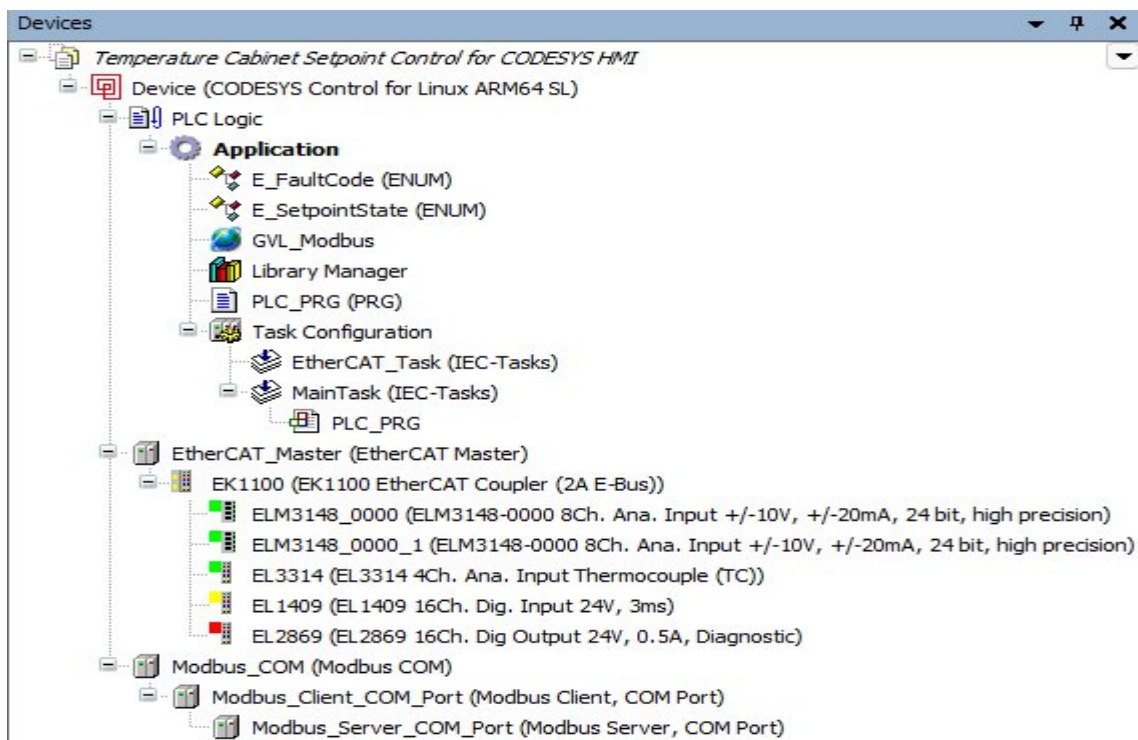
```

1      {attribute 'qualified_only'}
2      VAR_GLOBAL
3          (* FC03 cyclic read, offset 100 (fallback 99), length 1 -> actual temp raw (x10) *)
4          wReadTempValue    : WORD;
5          (* FC03 read-back, offset 300 (fallback 299), length 1 -> SP1 raw read (x10) *)
6          wSetpointlRead    : WORD;
7          (* FC06 write single register, offset 300 -> SP1 raw write (x10) *)
8          wSetpointlWrite   : WORD;
9          (* Rising-edge trigger var for the FC06 channel (I/O Mapping tab) *)
10         xWriteTrigger      : BOOL;
11         (* Master/slave diagnostic bits from the device IEC objects *)
12         xModbusError        : BOOL;  (* Modbus_Slave_COM.xError *)
13         xModbusDone         : BOOL;  (* last transaction done *)
14     END_VAR

```

2.3 — Device tree structure

In CODESYS, device management view, the tree looks like:



2.4 — Serial port settings (Modbus_COM)

Double-click Modbus_COM → General tab:

The 'Serial Port Configuration' dialog box shows the following settings:

Parameter	Value
COM port	2
Baud rate	19200
Parity	NONE
Data bits	8
Stop bits	1

Already verified the baud rate and com port for the Watlow F4S Temperature Controller according to **point 1.6** in this log and carried forward the same for CODESYS Integration

2.5 — Client settings (Modbus_Client_COM_Port)

Double-click **Modbus_Master_Client_COM_Port** → **General** tab:

Transmission mode is set to **RTU**

Response timeout and time between frames is set to **1000** and **50** respectively

Modbus RTU/ASCII

Transmission mode	☒ RTU ☐ ASCII
Response timeout (ms)	1000
Time between frames (ms)	50
☒ Auto-restart communication	



Bus Cycle Options

Bus cycle task	MainTask	Recreate required tasks
----------------	---------------------------------------	--

Critical observation: Bus cycle task MUST be set to **Main Task** explicitly. If it shows Use parent bus cycle setting or is empty, the channels will never run and will show red triangles with no error message.

2.6 — Server settings (Watlow_F4S)

Double-click **Modbus_Server_COM_Port**→ **General** tab:

Server address is set to **1** and response timeout is set to **1000**

Modbus RTU/ASCII

Server address [1..247]	1
Response timeout (ms)	1000



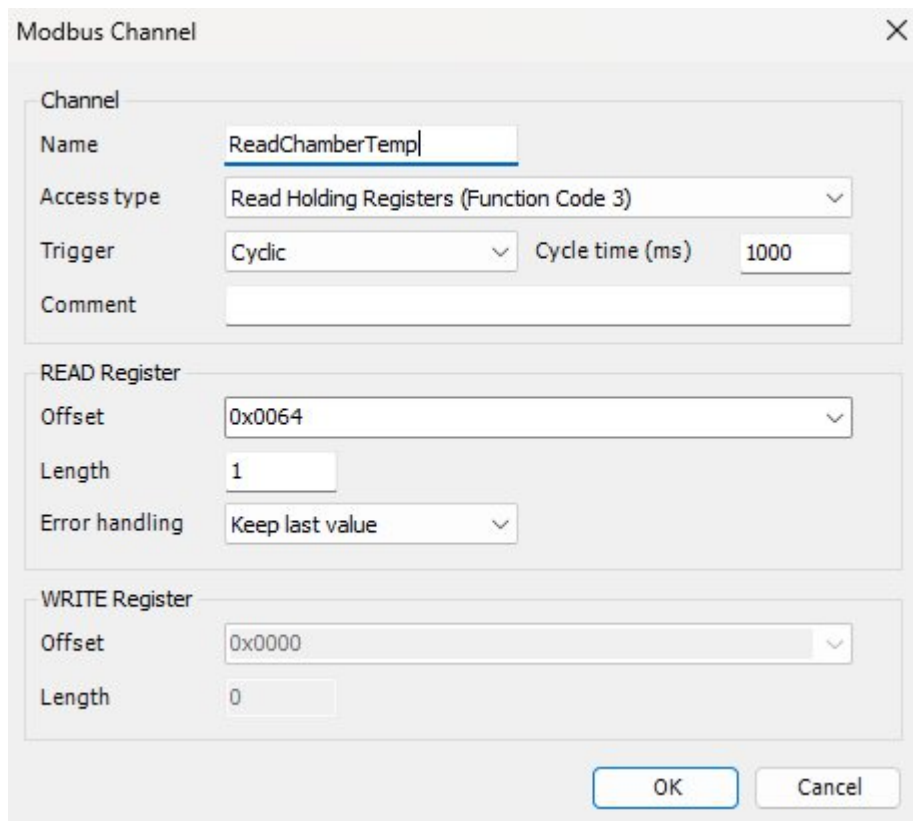
Verify same server address observed for the Watlow F4S Controller: -



2.7 — Create channels

On the **Watlow_F4S** slave, navigate to the **Channels** tab. Add three channels

Channel 0 — ReadChamberTemp:



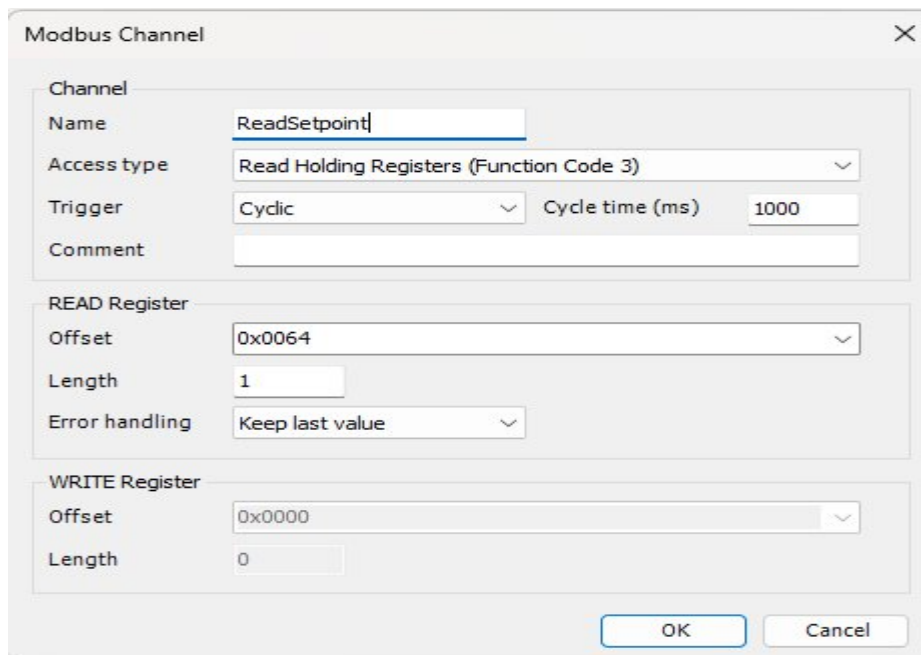
The image shows a 'Modbus Channel' configuration window for Channel 0. It has a title bar with a close button. The window is divided into three main sections: 'Channel', 'READ Register', and 'WRITE Register'. The 'Channel' section contains fields for Name ('ReadChamberTemp'), Access type ('Read Holding Registers (Function Code 3)'), Trigger ('Cyclic'), Cycle time (ms) ('1000'), and a Comment field. The 'READ Register' section contains fields for Offset ('0x0064'), Length ('1'), and Error handling ('Keep last value'). The 'WRITE Register' section contains fields for Offset ('0x0000') and Length ('0'). At the bottom right are 'OK' and 'Cancel' buttons.

Channel	
Name	ReadChamberTemp
Access type	Read Holding Registers (Function Code 3)
Trigger	Cyclic
Cycle time (ms)	1000
Comment	

READ Register	
Offset	0x0064
Length	1
Error handling	Keep last value

WRITE Register	
Offset	0x0000
Length	0

Channel 1 — ReadSetpoint:



The image shows a 'Modbus Channel' configuration window for Channel 1. It has a title bar with a close button. The window is divided into three main sections: 'Channel', 'READ Register', and 'WRITE Register'. The 'Channel' section contains fields for Name ('ReadSetpoint'), Access type ('Read Holding Registers (Function Code 3)'), Trigger ('Cyclic'), Cycle time (ms) ('1000'), and a Comment field. The 'READ Register' section contains fields for Offset ('0x0064'), Length ('1'), and Error handling ('Keep last value'). The 'WRITE Register' section contains fields for Offset ('0x0000') and Length ('0'). At the bottom right are 'OK' and 'Cancel' buttons.

Channel	
Name	ReadSetpoint
Access type	Read Holding Registers (Function Code 3)
Trigger	Cyclic
Cycle time (ms)	1000
Comment	

READ Register	
Offset	0x0064
Length	1
Error handling	Keep last value

WRITE Register	
Offset	0x0000
Length	0

Channel 2 — WriteSetpoint:

Modbus Channel

Channel

Name: WriteSetpoint

Access type: Write Single Register (Function Code 6)

Trigger: Rising edge Cycle time (ms): 1000

Comment:

READ Register

Offset: 0x0000

Length: 0

Error handling: Keep last value

WRITE Register

Offset: 0x012C

Length: 1

OK Cancel

Important takeaways from research: -

- ReadSetpoint on **Input Registers (FC04)** addresses a different memory space on the F4S where nothing exists. Every proven mbpoll read used -t 4 (holding registers, FC03).
- WriteSetpoint on **Cyclic** would rewrite register 300 (EEPROM-backed) on every cycle (~100 times/sec), destroying the EEPROM within hours. Rising edge = one write per operator request.

2.8 — I/O Mapping (Connect channels to GVL_Modbus)

On the **Watlow_F4S** slave, navigate to **I/O Mapping** tab.

For each channel, use the **browse button** (not free-text type) to select the existing GVL_Modbus variable:

1.Variable mapped to GVL Modbus to read the actual cabinet temperature: -

Variable	Mapping	Channel	Address	Type	Unit	Description
		CH0	%IW82	ARRAY [0..0] OF WORD		Read Holding Registers
 rChamberTemp		CH0[0]	%IW82	WORD		0x0064

2.Variable mapped to GVL Modbus to read the current setpoint on the screen: -

Variable	Mapping	Channel	Address	Type	Unit	Description
		CH1	%IW83	ARRAY [0..0] OF WORD		Read Holding Registers
 rConfirmedSetpoint		CH1[0]	%IW83	WORD		0x0064

3.Variable mapped to GVL Modbus to write the new desired setpoint for the operator:-

Variable	Mapping	Channel	Address	Type	Unit	Description
		Bit12	%IX167.4	BOOL		
		Bit13	%IX167.5	BOOL		
		Bit14	%IX167.6	BOOL		
		Bit15	%IX167.7	BOOL		
 xWriteTrigger		CH2	%QX2.0	BIT		Trigger variable
		CH2	%QW2	ARRAY [0..0] OF WORD		Write Single Register
 rReqSetPoint		CH2[0]	%QW2	WORD		0x012C

Critical detail observed: The **trigger row is separate from the data row**. The data row holds the value; the trigger Boolean row tells the channel "send it now." Both must be mapped and if only the data row is mapped, writes stage correctly but never fire.

Phase 3: PLC_PRG Implementation

3.1 — Create PLC_PRG in Application folder

Right-click **Application** → **Add POU** → **Program**, name it **PLC_PRG**.

Assign it to **Main Task** (right-click → **Task** menu).

3.2 — PLC_PRG code (without HMI, watch-window operation only)

1. Variable declaration

```
1  PROGRAM PLC_PRG
2  VAR
3      rChamberTemp      : REAL;
4      rReqSetpoint      : REAL := 26.5 ; (* set this in the watch window *)
5      rConfirmedSetpoint : REAL;
6
7      rMinSetpoint      : REAL := -40.0;
8      rMaxSetpoint      : REAL := 200.0;
9
10     eSetpointState     : E_SetpointState := E_SetpointState.IDLE;
11     eFaultCode         : E_FaultCode := E_FaultCode.NO_FAULT;
12
13     xStartWrite        : BOOL := FALSE; (* toggle this in the watch window *)
14     xPrevStartWrite    : BOOL := FALSE;
15     xWriteTriggered    : BOOL := FALSE;
16
17     dwWriteTimeout     : DWORD := 0;
18     dwMaxTimeout       : DWORD := 300; (* ~3 s at 10 ms MainTask *)
19 END_VAR
```

2. Execution pallette

```
1  (* --- 1. Scale raw -> engineering units (F4S carries one implied decimal) --- *)
2  rChamberTemp      := GVL_Modbus.wReadTempValue / 10.0;
3  rConfirmedSetpoint := GVL_Modbus.wSetpointlRead / 10.0 ;
4  rReqSetPoint      := GVL_Modbus.wSetpointlWrite / 10.0;
5
6  (* --- 2. Rising edge on the manual test trigger --- *)
7  xWriteTriggered := (xStartWrite AND NOT xPrevStartWrite);
8  xPrevStartWrite := xStartWrite;
9
10 (* --- 3. State machine --- *)
11 CASE eSetpointState OF
12
13     E_SetpointState.IDLE:
14         GVL_Modbus.xWriteTrigger := FALSE;
15         dwWriteTimeout := 0;
16         IF xWriteTriggered THEN
17             eSetpointState := E_SetpointState.READY;
```



```

18         END_IF
19
20     E_SetpointState.READY:
21         IF rReqSetpoint < rMinSetpoint THEN
22             eFaultCode := E_FaultCode.RANGE_LOW;
23             eSetpointState := E_SetpointState.FAULTED;
24         ELSIF rReqSetpoint > rMaxSetpoint THEN
25             eFaultCode := E_FaultCode.RANGE_HIGH;
26             eSetpointState := E_SetpointState.FAULTED;
27         ELSE
28             GVL_Modbus.wSetpointWrite := DWORD_TO_WORD(REAL_TO_DWORD(rReqSetpoint * 10.0));
29             eSetpointState := E_SetpointState.WRITING;
30         END_IF
31
32     E_SetpointState.WRITING:
33         GVL_Modbus.xWriteTrigger := TRUE;    (* one-scan pulse -> FC06 fires *)
34         dwWriteTimeout := 0;
35
36
37         eSetpointState := E_SetpointState.CONFIRM;
38
39     E_SetpointState.CONFIRM:
40         GVL_Modbus.xWriteTrigger := FALSE;    (* drop immediately to re-arm the edge *)
41         dwWriteTimeout := dwWriteTimeout + 1;
42
43         IF ABS(rConfirmedSetpoint - rReqSetpoint) < 0.1 THEN
44             eFaultCode := E_FaultCode.NO_FAULT;
45             eSetpointState := E_SetpointState.IDLE;
46         ELSIF dwWriteTimeout > dwMaxTimeout THEN
47             eFaultCode := E_FaultCode.NOT_ACCEPTED;
48             eSetpointState := E_SetpointState.FAULTED;
49         END_IF
50
51     E_SetpointState.FAULTED:
52         GVL_Modbus.xWriteTrigger := FALSE;
53         IF xWriteTriggered THEN
54
55
56             eFaultCode := E_FaultCode.NO_FAULT;
57             eSetpointState := E_SetpointState.READY;
58         END_IF
59     END_CASE

```

Why this design:

- xStartWrite is a **local variable** which is manually toggled from the watch window, isolated from the real hardware trigger.
- GVL_Modbus.xWriteTrigger is the **real, I/O-mapped trigger** which only the state machine controls it.
- One-scan pulse: trigger goes TRUE, next cycle goes FALSE. This guarantees exactly one FC06 per write request.
- xModbusError check catches real comms errors immediately, short-circuiting the timeout.

Phase 4: Build, Download, Verify

4.1 — Compile

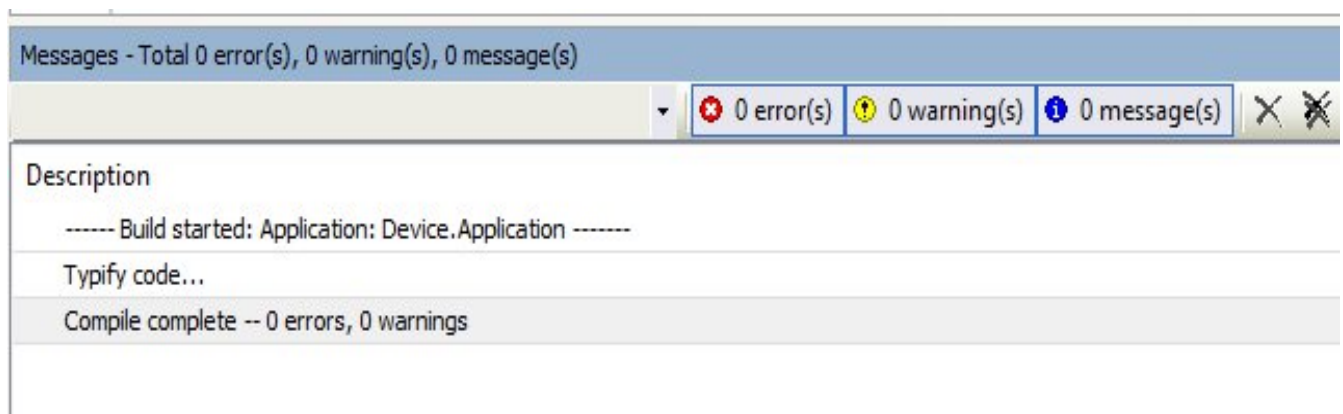
In CODESYS IDE:

Build → Rebuild All

Expected: 0 errors, 0 warnings.

If you see errors: Check that you used the **browse button** to map channel variables (not free-text type). CODESYS only accepts qualified names (Application.GVL_Modbus.xxx) when mapping to *existing* variables, not when creating new ones.

Output observed in CODESYS: -



4.2 — Download to Pi

Online menu → right-click the main **Device** → **Download**

Accept the "application differs, proceed anyway?" prompt.

Wait for download to complete (~10–20 seconds).

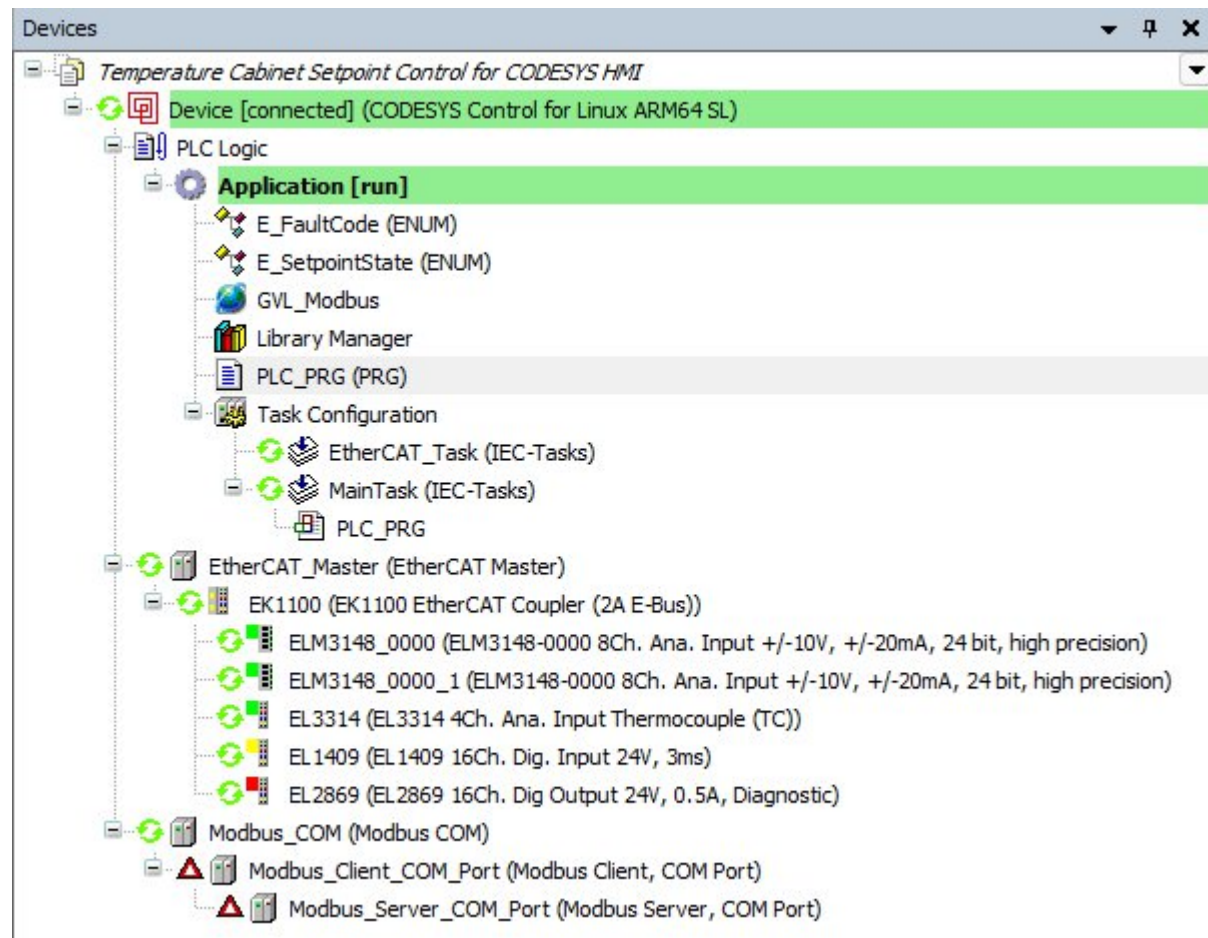
Expected Output Observed: -

```
mechatronics@LeftHandSmallTempCab:~/ssh/Temperature-Cabinet-Setpoint-Control-from-CODESYS-HMI $ ls -la /dev/ttyWatlowF4S
lrwxrwxrwx 1 root root 7 Jul 15 15:27 /dev/ttyWatlowF4S -> ttyUSB0
```

4.5 — Check device tree (green lights)

Look at the device tree on the left. Expand the device

Output Observed:- Modbus COM running but Modbus Client Server Not Running



Modbus COM is **running** the PCI-Bus and **not the serial port** which states the signal is transmitted from the Watlow F4S Controller but **not registered** onto CODESYS COM Port

SerialPort	:	Not running
PCI-Bus	:	Running

As the serial port is not running the watch window won't show anything :-

Watch 1					
Expression	Application	Type	Value	Prepared value	Execution point
 rChamberTemp	Device.Application	WORD	0		Cyclic Monitoring
 rConfirmedSetpoint	Device.Application	WORD	0		Cyclic Monitoring
 rReqSetPoint	Device.Application	WORD	0		Cyclic Monitoring
xWriteTriggered	Device.Application				Cyclic Monitoring

Troubleshooting by Symptom

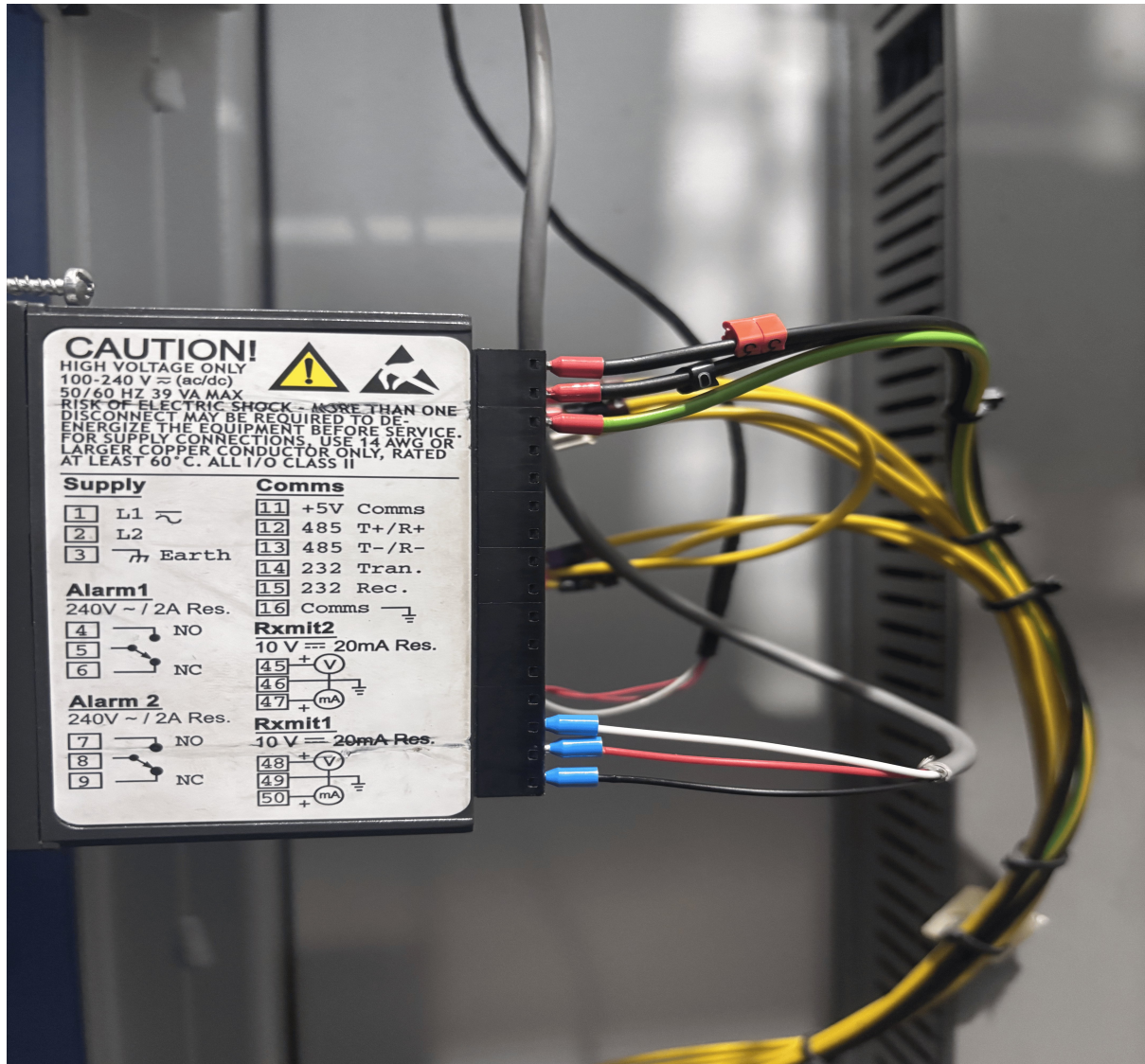
Symptom	Check	Fix
All devices RED on startup	Bus cycle task	Double-click Modbus_Master → General → "Bus cycle task" = MainTask (explicit, not "parent"). Rebuild + download.
rChamberTemp = 0 always	Channel mapping	Watlow_F4S → I/O Mapping → ReadChamberTemp → expand array → map [0] to wInput1Value (not the whole array).

Main problem	Why?	What to do instead
Do not leave mbpoll running when CODESYS is online	Port conflict so one process at a time	pskill -9 mbpoll before going online

My next recommendation: -

Try attempting to shift to RS485 Modbus Serial Communication which is also supported by the Watlow F4S Controller by doing the necessary research wiring and modbus linux integration followed by CodeSys modbus integration and test the results and do the necessary troubleshooting: -

Rear View Cabinet Controller Pins 12 and 13 supporting RS485 communication



Question: -

Should I move towards proving the same followed architecture for RS485 Modbus Serial Communication on these parameters or should I follow a different approach?